

Treinamento incremental aproximado de sistemas incorporados inteligentes

Projeto de Iniciação Científica Voluntária

8 de maio de 2024

Identificação

Projeto de Iniciação Científica Voluntária

Instituição de destino

Universidade Federal de Minas Gerais - Departamento de Engenharia Eletrônica - Laboratório de Sistemas Inteligentes.

Coordenador

Prof. Dr. Vitor Ângelo Maria Ferreira Torres
lattes: <http://lattes.cnpq.br/1801633347473015>

Orientadores

Prof. Dr. Vitor Ângelo Maria Ferreira Torres
lattes: <http://lattes.cnpq.br/1801633347473015>

Prof. Dr. Luiz Carlos Bambirra Torres
lattes: <http://lattes.cnpq.br/8559513813180335>

Prof. Dr. Frederico Gualberto Ferreira Coelho
lattes: <http://lattes.cnpq.br/5083061070631233>

Sumário

1	Contextualização	3
2	Objetivos	7
3	Plano de trabalho	8
4	Plano de orientação	9
5	Cronograma	9
6	Resultados Esperados	9

1 Contextualização

Otimização de Hardware

A regra que predominou por décadas desde o início da popularização dos computadores foi a de um aumento tão rápido e expressivo na velocidade de processamento e capacidade de armazenamento, que implementações otimizadas, em *hardware* e *software*, para aplicações específicas, foram deixando de justificar o esforço de desenvolvimento em muitos casos. Essa situação começou a mudar há pouco mais de uma década, como indica a Figura 1.

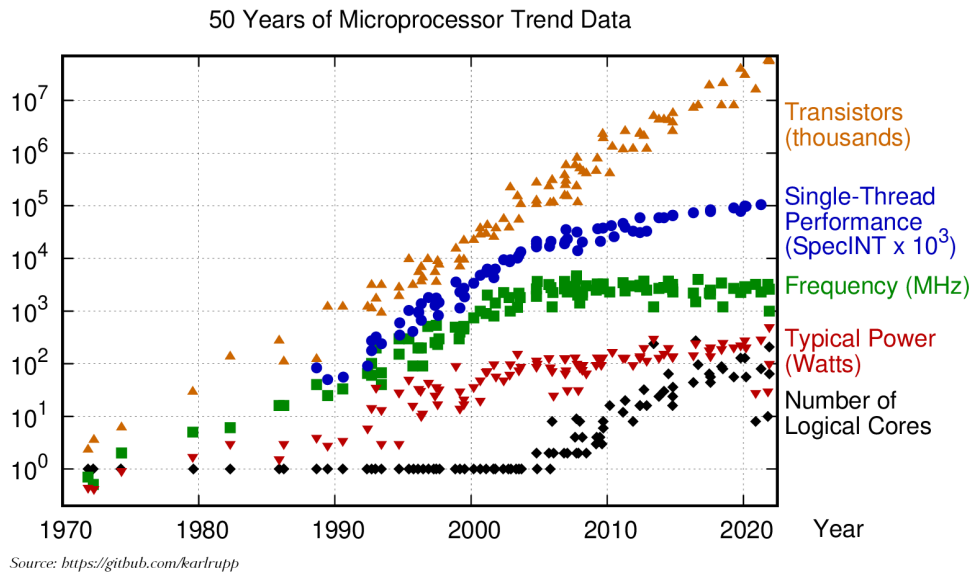


Figura 1: Fonte dos dados: “50 Years of Microprocessor Trend Data”, Karl Rupp, CC-BY 4.0.

A interrupção no rápido crescimento da frequência de *clock* dos processadores, que praticamente resultava em aumento de desempenho sem mudanças de arquitetura, foi deixando de ser a fonte mais direta de melhoria.

Mudanças nas arquiteturas de processamento foram sendo necessárias para se manter o aumento de desempenho por núcleo, como, por exemplo: implementações de *pipelines* cada vez mais complexos; execuções especulativas e paralelizadas em nível de instrução; criação de conjuntos de instruções cada vez maiores e retro-compatíveis. Toda essa complexidade não veio sem efeitos adversos: somente nos últimos anos já se conta na casa das dezenas a quantidade de problemas de segurança criados por falhas de projeto

em processadores. Diferentemente de um *software* que pode ser atualizado de forma simples, muitas vezes nem mesmo uma correção de microcódigo interno aos processadores resolve totalmente o problema, sendo necessária alguma mitigação com impacto no desempenho.

Ainda que o crescimento da densidade de integração dos circuitos (“Lei de Moore”) não tenha se estagnado, o que permitiu a criação de multiprocessadores com cada vez mais núcleos, a dissipação de calor começou a se tornar um obstáculo [EBSA⁺11], conforme indicado na Figura 2. Além da preocupação crescente com eficiência energética, isso traz impacto para sistemas que não podem contar com soluções de dissipação de calor mais robustas e para as plataformas alimentadas por baterias. Não é raro ver exemplos de sistemas que, mesmo contando com uma pequena ventilação forçada sob demanda, não conseguem sustentar seu desempenho de pico por mais que alguns segundos, tendo que recorrer, por exemplo, à redução dinâmica da frequência de *clock* do processador.

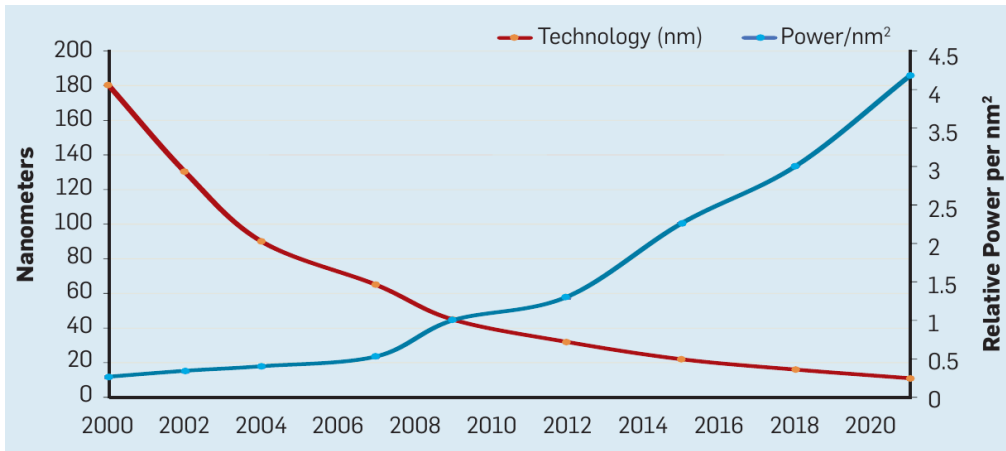


Figura 2: Dimensão de transistores e densidade de potência. Fonte: “*A new golden age for computer architecture*”, *Hennessy and Patterson*

O cenário de limitações é ainda agravado por um problema que antecede essa fase por mais de uma década [MV99] e continua se aprofundando: há uma distância crescente entre o desempenho dos processadores e das memórias externas. A solução para minimizar o tempo ocioso do processador tem sido a de tornar a hierarquia de memória cada vez mais complexa, com impactos tanto no consumo de energia quando em espaço ocupado no processador. Isso faz com que códigos perfeitamente compatíveis com uma determinada arquitetura possam ter desempenho aumentado por mais de uma ordem de

grandeza simplesmente por aproveitar a hierarquia de memória de forma mais eficiente. Adicione-se a isso o fato de que, além de haver um limite teórico para o aumento de desempenho por paralelismo [HM08], específico de cada aplicação, há uma disputa entre *threads* ao recurso compartilhado da memória (principal ou em um nível compartilhado de *cache*).

Todo esse cenário de dificuldades, descrito aqui de forma muito resumida, motivou os autores Hennessy e Patterson, em sua apresentação no recebimento do prêmio Turing, em Junho de 2018, a chamar o período atual de uma “Nova Era de Ouro para a Arquitetura de Computadores”. O conteúdo dessa palestra, detalhado em artigo [HP19], defende que o ritmo exponencial de aumento de desempenho com que os programadores se acostumaram nas últimas três décadas já acabou. Isso fica evidente até mesmo nas soluções comerciais já disponíveis, que lançam mão de aceleradores em *hardware* para as mais diversas aplicações como forma de entregar o desempenho necessário.

O projeto desses circuitos digitais específicos para determinadas tarefas, em oposição os que integram os processadores de uso geral, é um recurso usado desde o início da evolução digital exponencial. A fórmula é antiga e provada, pois desde o início da popularização dos computadores, co-processadores de ponto flutuante e aceleradores gráficos 2D, por exemplo, já eram usados como melhorias. O aumento da densidade de integração permitiu até mesmo que esses circuitos fossem sendo implementados na mesma pastilha. A mudança fundamental é que esse recurso de especialização tem se tornado uma forma de continuar evoluindo-se o desempenho dos processadores fabricados com o paradigma atual.

Aprendizado de Máquinas

As técnicas de aprendizado de máquinas têm mostrado resultados muito relevantes em plataformas de grande porte como, por exemplo: grandes modelos de linguagem (*ChatGPT*, *Gemini*), sistemas avançados de processamento de imagem para aplicações em tempo real (*Full Self Driving*) e sistemas de recomendação e retenção para grandes plataformas comerciais, sociais e de entretenimento.

Um efeito indireto do avanço da tecnologia de ponta que viabiliza essas aplicações é que o custo dos sistemas de gerações anteriores vai ficando cada vez menor. Plataformas de processamento que há 10 anos seriam consideradas de alto desempenho, hoje estão disponíveis em microcontroladores ou plataformas incorporadas (*Embedded Systems*) de baixo consumo e grande confiabilidade. Um dos exemplos dessa disponibilidade é a família de placas

Coral, baseada no acelerador TPU (*Tensor Processing Unit*), desenvolvido pela empresa Google, que tem um exemplo reproduzido na Figura 3.

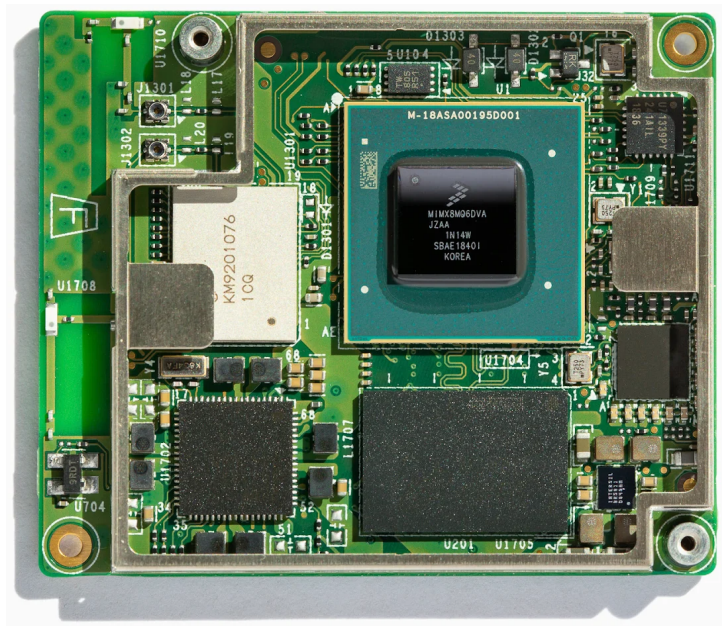


Figura 3: Exemplo de uma das soluções integradas *System On a Module* da plataforma Coral (Google Edge TPU, 4 TOPS (int8); 2 TOPS/W). Fonte: <https://coral.ai/products>

O processo de inferência das aplicações de aprendizado de máquinas é dominado por tarefas relativamente simples, como soma de produtos e convoluções, mas que têm grande impacto no tempo de processamento, devido à quantidade de operações necessárias. As várias gerações e versões da TPU conseguem, de forma muito eficiente e paralelizada, fazer um grande volume dessas operações rapidamente, até mesmo em plataformas portáteis.

Um exemplo de aplicação de muito impacto do aprendizado de máquinas, ainda razoavelmente recente, é o relógio inteligente da *Apple*. Entre tantos outros recursos, e em conjunto com um aplicativo de celular, o dispositivo é capaz de emitir alertas relacionados ao condicionamento cardíaco (como exemplificado na Figura 4) e ritmo irregular ou acima do esperado de batimentos. Há poucos anos foi publicado [PMH⁺19] um estudo com mais de 400 mil usuários do sistema, indicando uma concordância de 84% entre os alertas e as conclusões posteriores obtidas em atendimentos médicos utilizando-se eletrocardiogramas. Esses ainda são os primeiros passos de uma área que

pode trazer muitos impactos positivos, mas também problemas sérios se mal implementada ou mal utilizada.

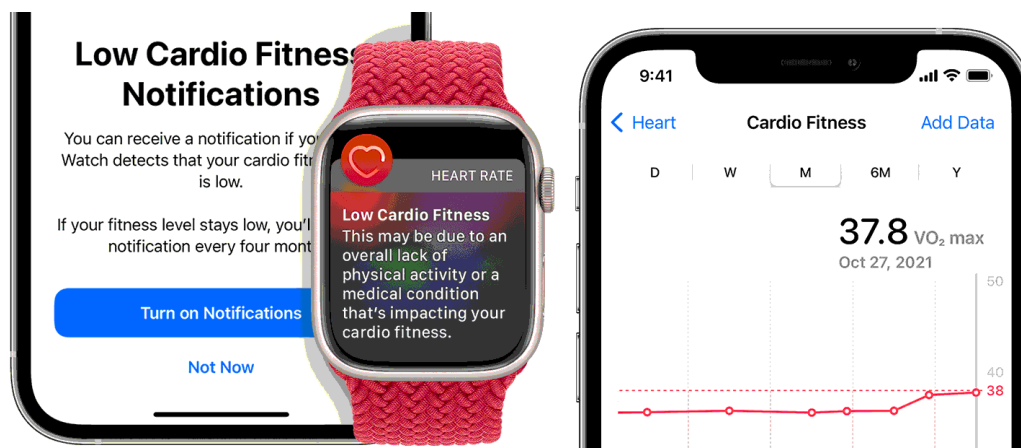


Figura 4: Exemplo de aviso sobre condicionamento cardíaco com o *Apple Watch*. Adaptado de: <https://support.apple.com/en-us/HT211856>

2 Objetivos

Objetivo geral

O paradigma atual de aprendizado de máquinas, ainda estruturalmente semelhante ao que se tornou popular na década de 80, pode ser separado em duas etapas. A etapa de aprendizado, dependendo do volume de dados e complexidade do sistema, pode ter uma demanda muito grande de recursos de *hardware* (memória e processamento). A etapa de inferência emprega o modelo criado no aprendizado para obter o comportamento desejado quando processar dados nunca vistos na primeira etapa. A inferência pode ser associada a uma aplicação final em que a plataforma de processamento precise ser muito eficiente, por exemplo pela necessidade de alimentação por baterias.

Esse projeto visa estudar plataformas otimizadas para o processo de inferência, aplicando conceitos de computação aproximada em arquiteturas dedicadas às operações específicas desse contexto. Uma vez que o processo de inferência está contido na etapa de aprendizado, deve-se buscar formas de permitir que essas arquiteturas otimizadas sejam usadas também em um processo de aprendizado incremental (de ajustes finos e adaptações do modelo criado). Essa necessidade é muito relevante para aplicações em que o

conhecimento básico, confiável, de um processo geral é obtido na etapa de aprendizado mas, na fase de aplicação final, esse modelo precisa ser adaptado a especificidades de cada sistema ou ajustado continuamente a mudanças lentas do processo.

Objetivos específicos

- Estudar e compreender os conceitos de aprendizado de máquina.
- Estudar e compreender os conceitos de computação aproximada.
- Estudar e compreender os conceitos de arquiteturas de computadores.
- Identificar aplicações para aprendizado incremental aproximado.
- Desenvolver os modelos e as aplicações definidas.
- Avaliar e testar protótipos com hardware real ou simulado.
- Prover uma iniciação ao meio acadêmico através do envolvimento com a pesquisa aplicada.
- Elaborar e publicar trabalhos técnicos e científicos.

3 Plano de trabalho

Os objetivos propostos deve ser alcançados com as seguintes etapas:

- Etapa 1: Revisão da literatura e estado da arte.
- Etapa 2: Desenvolvimento das aplicações.
 - Identificação de aplicações promissoras.
 - Seleção e proposta de algoritmos e arquiteturas.
 - Desenvolvimento do modelo definido.
- Etapa 3: Experimentos.
 - Medições de desempenho e melhorias.
 - Comparação da solução proposta com outras técnicas.
 - Elaboração das publicações adequadas.

4 Plano de orientação

A dinâmica de orientação dos(as) alunos(as) de IC é baseada em reuniões semanais, com a discussão das tarefas realizadas e o planejamento daquelas seguintes.

O local de desenvolvimento das atividades será o LSI (Laboratório de Sistemas Inteligentes) do Departamento de Engenharia Eletrônica, em colaboração com o LITC (Laboratório de Inteligência Computacional), do mesmo departamento. Os laboratórios dispõem de recursos computacionais para esse trabalho e, conforme a arquitetura escolhida, também de placas específicas para as aplicações finais.

5 Cronograma

Etapa / Mês	1	2	3	4	5	6	7	8	9	10	11	12
Etapa 1	X	X	X									
Etapa 2			X	X	X	X	X					
Etapa 3								X	X	X	X	X
Relatório Final												X

6 Resultados Esperados

Espera-se desse projeto o desenvolvimento de técnicas otimizadas de aprendizado de máquinas, especificamente para arquiteturas incorporadas e de baixo consumo. Tais técnicas devem ter foco em aplicações relevantes e recentemente viabilizadas pelos avanços nas plataformas de processamento focadas nesse contexto.

Deve-se buscar a expansão das colaborações com outro grupos de pesquisa, em especial para ampliar o leque de aplicações possíveis. Destaca-se, nesse sentido, a colaboração com o LabCOM (Laboratório de Redes, Tecnologias e Serviços de Comunicação a Distância), devido à grande correlação de aplicações incorporadas promissoras com o ambiente de “Internet das Coisas” (IoT) [DXHL14], cada vez mais pervasivo e acessível.

Espera-se também que os/as discentes adquiram ao longo do projeto um conjunto de habilidades relevantes tanto para a pesquisa acadêmica quanto para a aplicação prática do método científico.

Referências

- [DXHL14] Li Da Xu, Wu He, and Shancang Li. Internet of things in industries: A survey. *IEEE Transactions on industrial informatics*, 10(4):2233–2243, 2014.
- [EBSA⁺11] Hadi Esmailzadeh, Emily Blem, Renee St Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *ACM SIGARCH Computer Architecture News*, volume 39, pages 365–376. ACM, 2011.
- [HM08] Mark D Hill and Michael R Marty. Amdahl’s law in the multicore era. *Computer*, 41(7):33–38, 2008.
- [HP19] John L Hennessy and David A Patterson. A new golden age for computer architecture. *Communications of the ACM*, 62(2):48–60, 2019.
- [MV99] Nihar R. Mahapatra and Balakrishna Venkatrao. The processor-memory bottleneck: Problems and solutions. *XRDS*, 5(3es):2, 1999.
- [PMH⁺19] Marco V. Perez, Kenneth W. Mahaffey, Haley Hedlin, John S. Rumsfeld, Ariadna Garcia, Todd Ferris, Vidhya Balasubramanian, Andrea M. Russo, Amol Rajmane, Lauren Cheung, Grace Hung, Justin Lee, Peter Kowey, Nisha Talati, Divya Nag, Santosh E. Gummidipundi, Alexis Beatty, Mellanie True Hills, Sumbul Desai, Christopher B. Granger, Manisha Desai, and Mintu P. Turakhia. Large-scale assessment of a smartwatch to identify atrial fibrillation. *New England Journal of Medicine*, 381(20):1909–1917, 2019.